Lista: Matriz Flexível

AEDS 2 · Preparação para a Prova II · baseada nas questões das Provas II antigas · 19/10/2026

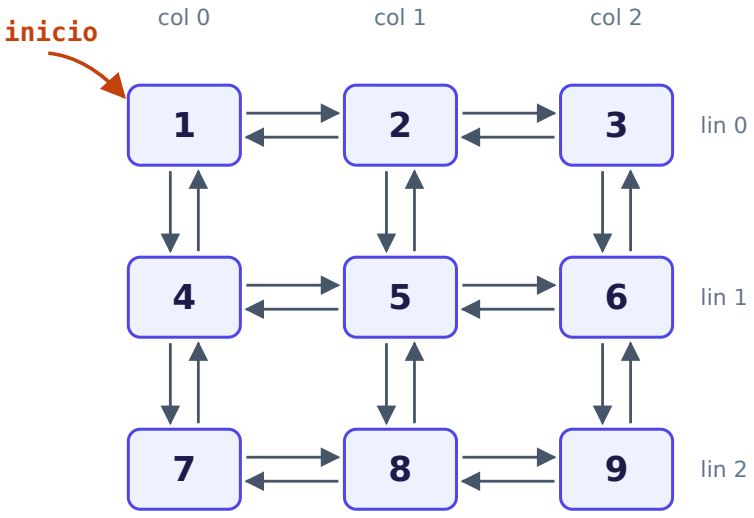
Como usar esta lista

- **Sem gabarito de propósito.** Faça no papel, desenhando caixinhas e setas, e me mande as respostas para correção.
- **Toda questão de código pede Θ justificada**, igual à prova.
- As questões marcadas com **[PROVA]** são versões de questões que já caíram. A **[PROVA ★]** é a `removerUltimaColuna`, a que mais zerou.
- Dificuldade: ●○○ aquecimento · ●●○ nível prova · ●●● desafio.
- Sempre que for escrever código, antes responda: *quais dedos (ponteiros auxiliares) eu preciso? Onde cada um começa? Para onde cada um anda?*

Em todas as questões, considere as classes do material:

```
class Celula { int elemento; Celula sup, inf, esq, dir; }
class Matriz { Celula inicio; int linha, coluna; }   // inicio = canto (0,0)
```

E esta matriz 3×3 como exemplo, quando a questão não disser outra:



Parte A: raciocínio sem escrever código

A1. Passeio pelas setas ●○○

Na matriz 3×3 acima, diga o que cada linha imprime. Se alguma linha der erro, diga **qual** erro e **em que ponto exato** da expressão ele acontece.

```
Celula x = inicio.dir.inf.inf;
System.out.println(x.elemento);           // (a)
x = x.esq.sup.dir.dir;
System.out.println(x.elemento);           // (b)
x = x.inf.esq.esq.sup.sup;
System.out.println(x.elemento);           // (c)
System.out.println(x.sup == null);         // (d)
x = x.inf.inf.inf.dir;
System.out.println(x.elemento);           // (e)
```

A2. Contando ponteiros ●●○

(a) Numa matriz 3×3, quantos ponteiros são **diferentes** de `null`? E quantos são `null`?
(b) Generalize para uma matriz l×c. Mostre como chegou na fórmula (dica: separe as ligações horizontais das verticais).
(c) Confira a sua fórmula com uma matriz 1×1 e com uma 1×4.

A3. Caça ao bug ●○○

O método abaixo deveria somar a coluna `j`. Ele compila, mas devolve a soma errada. Sem rodar: **qual valor ele devolve para `j = 1`** na matriz 3×3? Qual é o bug? Existe um segundo problema (pense em valores de `j` estranhos)?

```
public int somaColuna(int j) {
    Celula c = inicio;
    for (int k = 0; k < j; k++) c = c.dir;
    int soma = 0;
    while (c.inf != null) {
        soma += c.elemento;
        c = c.inf;
    }
    return soma;
}
```

A4. O preço do get ●●○

Uma colega somou todos os elementos de uma matriz n×n assim:

```
int soma = 0;
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
        soma += get(i, j);      // get desce i vezes e anda j vezes
```

(a) Quantos passos (`inf` + `dir`) o `get` dá no total? Monte o somatório e resolva.
(b) Qual é o Θ? Compare com o `mostrar()` dos dois laços.
(c) Para n = 100, quantas vezes mais lento é o jeito dela (aproximadamente)?

Parte B: percursos

B1. Diagonal secundária ●○○

`void mostrarDiagonalSecundaria()` : na 3×3, imprime 3, 5, 7. Exigência: **Θ(n)**. Lance exceção se a matriz não for quadrada. Antes do código: onde ela começa e qual é o "degrau"?

B2. Cobrinha ●●○

`void mostrarCobrinha()` : imprime a linha 0 da esquerda para a direita, a linha 1 da direita para a esquerda, a linha 2 da esquerda para a direita, e assim por diante. Na 3×3: `1 2 3 6 5 4 7 8 9` . Pergunta para pensar: quando termina uma linha, você está na **ponta** dela. Precisa voltar até a coluna 0 para descer? Ou dá para descer direto de onde está? Escreva a versão mais eficiente e diga o Θ.

B3. Imprimir a transposta ●●○

`void mostrarTransposta()` : imprime a matriz "deitada" (a coluna 0 vira a primeira linha impressa, e assim por diante). Na 3×3: `1 4 7 / 2 5 8 / 3 6 9` . Proibido usar `get` . Qual laço fica por fora e qual fica por dentro, comparado ao `mostrar()` ?

B4. É simétrica? ●●●

`boolean ehSimetrica()` : devolve `true` se `m[i][j] == m[j][i]` para todo i, j.
Pense: para comparar (i, j) com (j, i), você precisa de **dois dedos**. Se um anda pela **linha i para a direita** a partir da diagonal, o outro anda por onde? Desenhe o caminho dos dois na 3×3 para i = 0.
Faça sem `get` . Θ no melhor caso e no pior caso?

B5. Onde está o maior? ●●○

`int[] posicaoDoMaior()` : devolve um vetor `{i, j}` com a posição do maior elemento. Se houver empate, devolve o primeiro encontrado percorrendo linha por linha. Que informação extra você precisa carregar durante o percurso, já que a célula não sabe "em que posição ela está"?

Parte C: mudar a estrutura (estilo prova)

> **Checklist para toda questão desta parte**
> 1) Desenhe o ANTES e o DEPOIS. 2) Pinte de vermelho todo ponteiro que muda. 3) Para cada ponteiro pintado: liguei os dois lados? 4) Atualizei `linha` / `coluna`? 5) O `inicio` muda? 6) E se a matriz tiver só 1 linha ou só 1 coluna?

C1. Inserir coluna no fim ●●○

`void inserirColunaFim()`: acrescenta uma coluna de zeros à direita. Quantos dedos você precisa enquanto desce criando as células novas? Θ?

C2. Inserir linha no início ●●○

`void inserirLinhaInicio()`: acrescenta uma linha de zeros em cima da linha 0. O que acontece com o `inicio`? Em que momento exato você pode atualizá-lo sem perder a matriz?

C3. [PROVA ★] Remover a última coluna, em C ●●●

```c
typedef struct Celula { int elemento; struct Celula *sup, *inf, *esq, *dir; } Celula;
Celula *inicio;  int linha, coluna;
void removerUltimaColuna();   // se coluna == 1: printf de erro e não remove nada
```

Responda **antes** do código:
(a) Para remover a última coluna, em qual coluna o seu dedo tem que estar: na última ou na penúltima? E se usar a última, como faria?
(b) Para cada linha, liste os ponteiros que mudam e as células que recebem `free`.
(c) Qual a ordem certa entre "guardar quem desce", "cortar o `dir`" e "`free`"? O que dá errado se você fizer o `free` antes de descer?
(d) Θ? Separe o custo de **chegar** na coluna do custo de **remover**.
Depois, escreva o código completo.

C4. Remover a primeira linha (Java) ●●○

`void removerPrimeiraLinha()`. Duas pegadinhas para você descobrir: o que precisa acontecer com o `inicio`, e quais ponteiros da linha que **sobra** ainda apontam para as células removidas?

C5. Remover uma coluna qualquer ●●●

`void removerColuna(int j)`: remove a coluna j, que pode ser a primeira, uma do meio ou a última. Separe em casos. Qual caso é parecido com a C3? Qual caso mexe no `inicio`? No caso do meio, como a célula da esquerda e a da direita passam a se enxergar?

Parte D: combinações (como nas provas)

D1. [PROVA] Diagonal → lista dupla ●●○

Escreva `ListaDupla diagonalParaLista()` que devolve uma lista dupla **com célula cabeça** contendo os elementos da diagonal principal, na ordem (1, 5, 9). Não use um método `inserirFim` pronto: faça as ligações à mão.
Se a lista dupla não tivesse o ponteiro `ultimo`, o Θ mudaria? Para quanto?

D2. [PROVA] Matriz de listas: remover ímpares ●●●

Agora cada célula da matriz guarda uma **lista simples com célula cabeça** em vez de um `int`:

```
class No { int elemento; No prox; }
class Lista { No primeiro, ultimo; }              // primeiro = célula cabeça
class CelulaM { Lista lista; CelulaM sup, inf, esq, dir; }
```

`void removerImpares()`: remove todos os números ímpares de todas as listas da matriz.
(a) Quais são os 3 níveis de laço? (b) Na lista, remover exige ter a mão em quem? (c) Pegadinha: se o último nó de uma lista for removido, o que acontece com o ponteiro `ultimo`? (d) Θ em função de l, c e do tamanho máximo m de uma lista.

D3. [PROVA] Diagonal unificada ●●●

Usando a mesma matriz de listas da D2 (quadrada), escreva `Lista diagUnificada()`, que devolve **uma única lista nova** com todos os elementos das listas da diagonal principal, na ordem (primeiro a lista de (0,0), depois a de (1,1), ...). As listas originais **não** podem ser alteradas. Por que "emendar" as listas (fazer o `ultimo.prox` de uma apontar para a próxima) quebraria essa regra?

D4. [PROVA] Matriz + árvore ●●○

(a) `void ordenarComArvore()`: insere todos os elementos da matriz numa **árvore binária de pesquisa** e depois imprime em ordem crescente. Qual caminhamento você usa?
(b) Dê o Θ no melhor e no pior caso. Que formato de matriz gera o pior caso? Dê um exemplo 2×3.
(c) ●●● Agora, em vez de imprimir, **reescreva** a matriz já ordenada (linha por linha). Dica: o caminhamento da árvore e o percurso da matriz precisam andar "juntos". Como passar o dedo da matriz para a recursão?

Parte E: verdadeiro ou falso (justifique!)

Lembre

"Falso" sem justificativa não vale ponto. Ao refutar, mostre o valor certo ou um contraexemplo.

1. Na matriz flexível, acessar a posição (i, j) custa Θ(1), assim como no array `int[][]`.
2. Imprimir a diagonal principal de uma matriz flexível n×n pode ser feito em Θ(n).
3. `removerUltimaColuna` numa matriz l×c é Θ(l), pois só remove uma célula por linha.
4. Uma matriz flexível l×c tem exatamente 4·l·c ponteiros diferentes de `null`.
5. Inserir uma linha no **fim** da matriz exige alterar o ponteiro `inicio`.
6. Para percorrer a matriz inteira, é obrigatório voltar ao `inicio` no começo de cada linha.
7. Se esquecermos de ligar os ponteiros `inf` no construtor (só ligando `sup`), o `mostrar()` dos dois laços ainda imprime a matriz toda.

Sugestão de ordem

Dia 1: Parte A inteira + B1 e B2. Dia 2: C1 a C4 (a C3 cronometrada em 30 min, sem consultar). Dia 3: D1 a D4 + Parte E. As ●●● (B4, C5, D2, D3, D4c) são as que mais treinam o raciocínio: se travar, me pede uma dica em vez da resposta.